

S3 & CloudFront

Object Storage · CDN · Security · Lifecycle · Lambda@Edge

AWS Tutorial · ShopFlow

05-00

Chapter Overview

Topic	AWS Service	ShopFlow Use-Case
S3 Storage Classes	Standard, IA, Glacier, Intelligent-Tiering	Product images Standard → IA after 90 days
S3 Static Website	S3 + CloudFront + Route53	React SPA hosted in S3 behind CloudFront
S3 Security	Bucket policies, ACLs, S3 Block Public Access	All assets private — CloudFront OAC only
S3 Lifecycle Rules	Transition + Expiration rules	Auto-tier product images, delete tmp files 7d
S3 Event Notifications	SQS/SNS/Lambda triggers on object events	Trigger thumbnail generation on upload
S3 Replication	CRR (cross-region) + SRR (same-region)	CRR to us-west-2 for disaster recovery
S3 Performance	Multipart, S3 Transfer Acceleration, pre-signed URLs	Multipart for images > 5MB
CloudFront CDN	Edge locations, Origins, Behaviors, Cache Policies	600+ POPs serve ShopFlow global customers
CloudFront Security	WAF integration, Geo Restriction, Signed URLs	Signed URLs for purchase receipts, WAF on API
CloudFront Functions	Viewer Request/Response functions (JS)	JS test, URL normalization at edge
Lambda@Edge	Viewer/Origin Request/Response (Node.js)	JS header injection, dynamic resizing
S3 Cost Optimization	Intelligent-Tiering, Glacier Deep Archive	Save 73% on archived order attachments

05-01

S3 Core Concepts — Buckets, Objects & Consistency

Amazon S3 is ShopFlow's primary object store for product images (12M objects, 4.2TB), static React SPA assets (220MB), order PDF receipts (1.8M docs/year), CloudWatch logs archive, and Aurora automated backups. S3 provides 11 nines (99.999999999%) durability — data is replicated across ≥3 AZs within a region.

S3 Core Concepts

Concept	Description	ShopFlow Example
Bucket	Globally unique container for objects. Region-scoped.	shopflow-product-images-prod (us-east-1)
Object	File + metadata. Max 5TB. Key is full path.	products/electronics/iphone15/main.jpg
Object Key	Unique identifier within a bucket (the full path).	receipts/2024/11/29/order-8827361.pdf
Versioning	Keep all versions of an object. Protects against accidental deletion.	Enabled on product-images bucket (rollback)
Consistency Model	Strong read-after-write for all operations (since 2020).	PUT completes, GET immediately returns new object
Storage Class	Tier that determines durability, availability, and price.	Standard for active images, IA after 90 days
Prefix	String before the first "/" — NOT a real directory.	products/electronics/ — shard by prefix for performance
ETag	MD5 hash of object content — used for conditional requests.	Client uses ETag for validation

ShopFlow S3 Bucket Inventory

Bucket Name	Contents	Size	Storage Class	Versioning
shopflow-product-images-prod	All product photos, thumbnails	4.2 TB	Standard → IA (90d)	Yes
shopflow-spa-assets-prod	React build: JS, CSS, HTML, fonts	220 MB	Standard	No
shopflow-order-receipts-prod	PDF receipts per order	890 GB (growing)	Standard → Glacier (1yr)	Yes
shopflow-logs-archive-prod	ALB + CloudFront access logs	2.1 TB / year	Standard → Glacier IA (300d)	No
shopflow-db-backups-prod	Aurora automated snapshots export	1.8 TB	Standard → Deep Archive (90d)	No
shopflow-cf-templates-prod	CDK / CloudFormation templates	15 MB	Standard	Yes

■ Use CloudFront Origin Access Control (OAC) — not presigned URLs and not public buckets — for serving S3 content. OAC requires no client-side signing, is IAM-controlled, and supports all S3 request types including SSE-KMS objects.

05-02

S3 Storage Classes — Cost vs Access Tradeoff

S3 offers 6 storage class tiers with different durability, availability, access latency, and price. Choosing the right class for each object's access pattern is the single biggest S3 cost lever. ShopFlow saves ~\$580/month by tiering rarely-accessed content.

Storage Class	Durability	Availability	Min Duration	\$/GB/mo	Retrieval Fee	ShopFlow Usage
S3 Standard	11 nines	99.99% (3 AZ)	None	\$0.023	None	Active product images, SPA assets
S3 Intelligent-Tiering	11 nines	99.9%	None	\$0.023 + \$0.0025/1k obj/mo	None	Product images with unknown access patterns
S3 Standard-IA	11 nines	99.9% (3 AZ)	30 days	\$0.0125	\$0.01/GB	Product images 90-365 days old
S3 One Zone-IA	11 nines	99.5% (1 AZ only)	30 days	\$0.01	\$0.01/GB	Reproducible data (thumbnails, static assets)
S3 Glacier Instant	11 nines	99.9%	90 days	\$0.004	\$0.03/GB	Order receipts older than 1 year
S3 Glacier Flexible	11 nines	99.99%	90 days	\$0.0036	\$0.01-\$0.03/GB, 1-12hr	Order attachments 2+ years old
S3 Glacier Deep Archive	11 nines	99.99%	180 days	\$0.00099	\$0.02/GB, 12-48hr	DB backups beyond 90 days —

Lifecycle Policy — CDK

```
const imagesBucket = new s3.Bucket(this, "ProductImages", {
  bucketName: "shopflow-product-images-prod",
  versioned: true,
  encryption: s3.BucketEncryption.S3_MANAGED,
  blockPublicAccess: s3.BlockPublicAccess.BLOCK_ALL,
  lifecycleRules: [{
    id: "ImageTiering",
    enabled: true,
    transitions: [
      {storageClass: s3.StorageClass.INFREQUENT_ACCESS,
       transitionAfter: Duration.days(90)},
      {storageClass: s3.StorageClass.GLACIER_INSTANT_RETRIEVAL,
       transitionAfter: Duration.days(365)},
    ],
    noncurrentVersionsToRetain: 3,
    noncurrentVersionTransitions: [{
      storageClass: s3.StorageClass.INFREQUENT_ACCESS,
      transitionAfter: Duration.days(30),
    }],
  }, {
    id: "DeleteIncompleteMultipart",
    abortIncompleteMultipartUploadAfter: Duration.days(7),
  }],
});
```

■ Enable S3 Intelligent-Tiering for product images where access patterns are uncertain. It monitors 30-day access frequency automatically and moves objects between Frequent Access (Standard price) and Infrequent Access (IA price) tiers with no retrieval fees and no minimum duration penalty.

ShopFlow enforces a "zero public S3" policy. Every bucket has Block Public Access enabled at both account and bucket level. All content delivery goes through CloudFront with Origin Access Control (OAC), which cryptographically signs every S3 request using the CloudFront distribution's IAM-equivalent identity.

ShopFlow S3 Zero-Trust Access Model

Account Level: S3 Block Public Access = ALL BLOCKED

Blocks any policy or ACL that grants public access

Enabled via: `aws s3control put-public-access-block --account-id 12345678901`

Bucket Level: Block Public Access + no public bucket policy

`blockPublicAcls: true` — ignore PublicRead ACL grants

`blockPublicPolicy: true` — reject policies that grant public access

`ignorePublicAcls: true` — ignore all ACL grants to public

`restrictPublicBuckets: true` — restrict cross-account public policies

Content Access: CloudFront OAC → S3 only (never direct)

OAC identity: `cloudfront.amazonaws.com` service principal

Bucket policy: Allow only `cloudfront.amazonaws.com` with matching distributi

Authenticated Access: Presigned URLs with 15-minute TTL

Order receipts: Lambda generates presigned URL, returns in API response

Admin uploads: S3 presigned POST URL (5-minute TTL for frontend upload)

OAC Bucket Policy (CDK-generated)

```
imagesBucket.addToResourcePolicy(new iam.PolicyStatement({
  effect: iam.Effect.ALLOW,
  principals: [new iam.ServicePrincipal("cloudfront.amazonaws.com")],
  actions: ["s3:GetObject"],
  resources: [imagesBucket.arnForObjects("*")],
  conditions: {
    "StringEquals": {
      "AWS:SourceArn": cfDistribution.distributionArn
    }
  }
}));
```

S3 Presigned URL — Lambda Handler

```
// Generate presigned URL for order receipt download
const command = new GetObjectCommand({
  Bucket: "shopflow-order-receipts-prod",
  Key: `receipts/${year}/${month}/${orderId}.pdf`,
});
const presignedUrl = await getSignedUrl(s3Client, command, {
  expiresIn: 900, // 15 minutes – order receipt is a one-time download
```

```
});  
return { statusCode: 200, body: JSON.stringify({ downloadUrl: presignedUrl }) };
```

■ Never log S3 presigned URLs — they are bearer tokens. Anyone with the URL can access the object until expiry. Always set short expiry (5-15 minutes), log only the object key, and use bucket policy Conditions to restrict to specific source IPs if possible.

05-04

S3 Event Notifications — Thumbnail Generation Pipeline

S3 Event Notifications trigger SQS, SNS, or Lambda functions when objects are created, deleted, or restored. ShopFlow uses S3 → SQS → Lambda to generate product image thumbnails in 4 sizes when sellers upload new images.

ShopFlow Product Image Processing Pipeline

Step 1: Seller uploads product image via Admin Portal

Frontend gets presigned POST URL from API (5-min TTL)

Browser uploads directly to S3 (bypasses API servers, no size limit)

Step 2: S3 emits ObjectCreated event to SQS queue

Prefix filter: products/originals/* (only originals trigger)

Event types: s3:ObjectCreated:Put, s3:ObjectCreated:CompleteMultipartUpload

Step 3: Lambda processes SQS batch (max 10 images/invoke)

Download original from S3 (products/originals/prod-123/main.jpg)

Generate 4 thumbnails: 1200x1200, 800x800, 400x400, 100x100 (thumbnails)

Upload 4 thumbnails to S3 (products/thumbnails/prod-123/main-*.jpg)

Update DynamoDB: set product.imageStatus = PROCESSED

Step 4: CloudFront serves thumbnails globally via edge cache

CDN URL: <https://cdn.shopflow.com/products/thumbnails/prod-123/main-400.jpg>

Cache-Control: max-age=31536000 (1 year — immutable after processing)

S3 Event Notification — CDK

```
import * as s3n from "aws-cdk-lib/aws-s3-notifications";  
imagesBucket.addEventNotification(  
  s3.EventType.OBJECT_CREATED,  
  new s3n.SqsDestination(thumbnailQueue),  
  { prefix: "products/originals/", suffix: ".jpg" },  
);  
imagesBucket.addEventNotification(  
  s3.EventType.OBJECT_CREATED,  
  new s3n.SqsDestination(thumbnailQueue),  
  { prefix: "products/originals/", suffix: ".png" },  
);  
// Lambda triggered by SQS (batch window: 30 seconds, batch size: 10)  
thumbnailFn.addEventSource(new lambdaEvents.SqsEventSource(thumbnailQueue, {  
  batchSize: 10,
```

```
maxBatchingWindow: Duration.seconds(30),  
reportBatchItemFailures: true, // partial batch failure support  
}});
```

■ Use SQS as buffer between S3 events and Lambda processing. Direct S3 → Lambda can overwhelm Lambda concurrency during bulk uploads (sellers sometimes upload 500 images at once). SQS smooths the burst, and Lambda scales at a safe rate.

Amazon CloudFront is a global CDN with 600+ edge locations. ShopFlow uses CloudFront as the single entry point for all web traffic — the SPA, product images, APIs, and WebSocket connections — enabling global sub-50ms latency for static content and reducing ALB traffic by 85%.

CloudFront Request Lifecycle

CloudFront Request Flow — ShopFlow

User Browser → DNS → Route53 → shopflow.com → CF Edge POP

CloudFront Edge: check cache by cache key (URL + headers)

Cache HIT: serve from edge — sub-5ms, zero origin load

Cache MISS: forward to origin based on behavior path pattern

Behavior Routing (by path pattern, in priority order):

/api/* → Origin: ALB (EC2/Fargate) — never cached (TTL=0)

/*.js, /\.css, /\.png → Origin: S3 SPA bucket — cache 1yr

/products/images/* → Origin: S3 images bucket — cache 7 days

/ws/* → Origin: ALB WebSocket — no cache, HTTP/2 upgrade

Default (/) → Origin: S3 SPA (index.html) — cache 60s

Response: CloudFront adds X-Cache header (HIT/MISS), compresses (gzip/br)

CloudFront Distribution — CDK

```
const distribution = new cloudfront.Distribution(this, "ShopFlowCF", {
  comment: "shopflow.com CDN — production",
  defaultRootObject: "index.html",
  domainNames: ["shopflow.com", "www.shopflow.com"],
  certificate: acmCert,
  minimumProtocolVersion: cloudfront.SecurityPolicyProtocol.TLS_V1_2_2021,
  enableLogging: true,
  logBucket: logsBucket,
  logFilePrefix: "cf-access-logs/",
  // Default behavior: React SPA from S3
  defaultBehavior: {
    origin: new origins.S3BucketOrigin(spaBucket, {
      originAccessControl: oac }, // OAC (not OAI — deprecated)
    viewerProtocolPolicy: cloudfront.ViewerProtocolPolicy.REDIRECT_TO_HTTPS,
    cachePolicy: cloudfront.CachePolicy.CACHING_OPTIMIZED,
    compress: true,
  },
  additionalBehaviors: {
    "/api/*": {
      origin: new origins.LoadBalancerV2Origin(alb, {
        protocolPolicy: cloudfront.OriginProtocolPolicy.HTTPS_ONLY },
      cachePolicy: cloudfront.CachePolicy.CACHING_DISABLED,
      allowedMethods: cloudfront.AllowedMethods.ALLOW_ALL, // pass POST/PUT/DELETE
      viewerProtocolPolicy: cloudfront.ViewerProtocolPolicy.HTTPS_ONLY,
    },
  },
});
```

```
},  
});
```

■ Use `CachePolicy.CACHING_DISABLED` for `/api/*` (TTL=0, no caching) but still benefit from CloudFront: global anycast routing routes requests to the nearest AWS edge, then AWS backbone to your ALB — faster than public internet even for dynamic content.

05-06

CloudFront Security — WAF, Signed URLs & Geo Restriction

CloudFront integrates with AWS WAF for application-layer protection, supports Signed URLs and Signed Cookies for private content, and offers Geo Restriction to block traffic from specific countries.

CloudFront Security Layers

Security Layer	What It Blocks	ShopFlow Config
AWS WAF on CloudFront	SQL injection, XSS, rate limiting, bot traffic	AWSManagedRulesCommonRuleSet + custom rate rules
HTTPS Only (minimum TLS 1.2)	SSL/TLS downgrade attacks, BEAST, POODLE	TLS_V1_2_2021 minimum policy — drops TLS 1.0/1.1
CloudFront Signed URLs	Unauthorized access to private S3 content	Order receipts: CF signed URL, 15-min TTL
Geo Restriction (block list)	Traffic from specific countries (export control)	Block embargoed countries — OFAC compliance
Origin Shield	Reduces origin load, adds caching layer	Enabled for S3 origin in us-east-1
Field-Level Encryption	Protects sensitive form fields end-to-end	Payment form: RSA encrypt card number at edge

WAF WebACL — CDK (CloudFront scope)

```
// WAF rules for CloudFront MUST be created in us-east-1 (global scope)  
const webAcl = new wafv2.CfnWebACL(this, "ShopFlowWAF", {  
  scope: "CLOUDFRONT", // must be CLOUDFRONT or REGIONAL  
  defaultAction: {allow: {}},  
  rules: [{  
    name: "AWSCommonRules",  
    priority: 1,  
    overrideAction: {none: {}},  
    statement: {managedRuleGroupStatement: {  
      vendorName: "AWS",  
      name: "AWSManagedRulesCommonRuleSet"}},  
    visibilityConfig: {sampledRequestsEnabled:true,  
      cloudWatchMetricsEnabled:true,  
      metricName:"CommonRules"},  
  }, {  
    name: "CheckoutRateLimit",  
    priority: 2,  
    action: {block: {}},  
    statement: {rateBasedStatement: {  
      limit: 100, aggregateKeyType:"IP",  
      scopeDownStatement: {byteMatchStatement:{  
        fieldToMatch:{uriPath:{}},  
        positionalConstraint:"STARTS_WITH",  
        searchString:"/api/v1/orders",  
        textTransformations:[{priority:0,type:"NONE"}]}},  
      visibilityConfig:{sampledRequestsEnabled:true,  
        cloudWatchMetricsEnabled:true,metricName:"CheckoutRateLimit"},  
    }},  
    visibilityConfig:{sampledRequestsEnabled:true,  
      cloudWatchMetricsEnabled:true,metricName:"ShopFlowWAF"},  
  }],  
});
```

■ WAF for CloudFront MUST be created in us-east-1 region — CloudFront is a global service tied to us-east-1. Even if your application is in us-west-2, create the WebACL stack in us-east-1 with `env: { region: "us-east-1" }` in CDK.

Lambda@Edge runs Node.js or Python functions at CloudFront edge locations — reducing latency for dynamic processing that'd otherwise require a round-trip to the origin. ShopFlow uses Lambda@Edge for auth header injection and dynamic image resizing.

Lambda@Edge Trigger Points

Trigger	Timing	Latency Impact	Max Execution Time	ShopFlow Use
Viewer Request	After CF receives request, before cache check	Highest impact — runs every request (cache miss+hit)	5 seconds	URL normalization: /products → /pr
Origin Request	Before request goes to origin (cache miss only)	Medium — cache misses only	30 seconds	Inject auth header for ALB origin va
Origin Response	After origin responds, before CF caches it	Medium — cache misses only	30 seconds	Add security headers (HSTS, CSP)
Viewer Response	After CF returns to viewer (every request)	Highest — runs every response	5 seconds	Not used (use Origin Response ins

Lambda@Edge — Auth Header Injection (Origin Request)

```
// TypeScript Lambda@Edge - Origin Request trigger
// Injects X-CloudFront-Secret header to verify request came via CF, not direct
export const handler = async (event: CloudFrontRequestEvent) => {
  const request = event.Records[0].cf.request;
  // Inject shared secret that ALB security group only trusts from CF
  request.headers['x-cloudfront-secret'] = [{
    key: 'X-CloudFront-Secret',
    value: process.env.CF_SECRET! // from Lambda@Edge env (us-east-1 only)
  }];
  // For image paths, rewrite to optimal format based on Accept header
  if (request.uri.startsWith('/products/images/')) {
    const acceptHeader = request.headers['accept']?.[0]?.value ?? '';
    if (acceptHeader.includes('image/webp')) {
      request.uri = request.uri.replace(/\.jpg$/, '.webp'); // serve WebP
    }
  }
  return request;
};
```

CloudFront Functions vs Lambda@Edge — Quick Choice Guide

Feature	CloudFront Functions	Lambda@Edge
Languages	JavaScript (ES5)	Node.js, Python
Max execution time	1ms (!!)	5-30 seconds
Access to network/filesystem	No — pure computation only	Yes (DynamoDB, Secrets Manager)
Price vs Lambda@Edge	~10x cheaper	Standard Lambda pricing
Trigger points	Viewer Request/Response only	All 4 trigger points
Use for	URL rewrite, header manipulation, A/B routing	Auth, DB lookup, image transform

■ Use CloudFront Functions for simple request manipulation (URL normalization, header add/remove, A/B testing by cookie). Use Lambda@Edge only when you need network access or >1ms execution time. CloudFront Functions cost \$0.10/million vs Lambda@Edge at \$0.60/million.

S3 throughput scales with prefix diversity and upload strategy. For ShopFlow's large image uploads (some RAW product photos are 50MB+), multipart upload is required for files over 5GB and recommended for files over 5MB.

Multipart Upload — When and How

File Size	Recommended Strategy	Benefit
< 5 MB	Single PUT upload	Simpler, lower overhead for small files
5 MB - 100 MB	Multipart upload (10 MB parts)	Retry individual parts, parallel upload
100 MB - 1 GB	Multipart upload (25 MB parts)	4 parallel streams = 4x throughput
> 1 GB	Multipart + S3 Transfer Acceleration	TA routes via AWS edge → backbone, much faster from far regions
> 5 GB (required)	Multipart is mandatory — single PUT max is 5GB S3 API requirement, not optional	

Multipart Upload — Spring Boot

```
// Upload product image with multipart (Spring Boot + AWS SDK v2)
@Service
public class ProductImageService {
    private final S3Client s3;
    private static final int PART_SIZE = 10 * 1024 * 1024; // 10 MB parts
    public String uploadProductImage(String productId, InputStream imageStream,
    long fileSize, String contentType) {
        String key = "products/originals/" + productId + "/main.jpg";
        CreateMultipartUploadResponse init = s3.createMultipartUpload(
        r -> r.bucket("shopflow-product-images-prod").key(key)
        .contentType(contentType)
        .serverSideEncryption(ServerSideEncryption.AES256));
        List<CompletedPart> parts = new ArrayList<>();
        byte[] buffer = new byte[PART_SIZE];
        int partNumber = 1; int bytesRead;
        while ((bytesRead = imageStream.read(buffer)) > 0) {
            UploadPartResponse part = s3.uploadPart(
            r -> r.bucket("shopflow-product-images-prod").key(key)
            .uploadId(init.uploadId()).partNumber(partNumber),
            RequestBody.fromBytes(Arrays.copyOf(buffer, bytesRead)));
            parts.add(CompletedPart.builder().partNumber(partNumber++)
            .eTag(part.eTag()).build());
        }
        s3.completeMultipartUpload(r -> r
        .bucket("shopflow-product-images-prod").key(key)
        .uploadId(init.uploadId())
        .multipartUpload(m -> m.parts(parts)));
        return "https://cdn.shopflow.com/" + key;
    }
}
```

■ Add an S3 lifecycle rule to abort incomplete multipart uploads after 7 days: `abortIncompleteMultipartUploadAfter: Duration.days(7)`. Failed uploads that don't get completed leave orphaned parts that incur storage charges indefinitely.

S3 Cross-Region Replication (CRR) asynchronously copies every new object from a source bucket to a destination bucket in a different region. ShopFlow replicates product images to us-west-2 for disaster recovery — RTO <15 minutes for image serving failover.

CRR vs SRR — When to Use Each

Replication Type	Source → Destination	Latency	Use Case	Cost
Cross-Region (CRR)	Different AWS regions	Minutes (async, best-effort)	Disaster recovery, compliance, multi-region serving	Storage in both regions + transfer
Same-Region (SRR)	Same region, different bucket	Seconds (async)	Log aggregation, account separation, testing	Storage in both buckets, no transfer
S3 Batch Replication	Any → any (existing objects)	Hours to days	Backfill existing objects that predate replication	Per-object fee (\$0.25/million objects replicated)
Bi-directional CRR	Region A ↔ Region B (both directions)	Minutes each way	Active-active multi-region (requires configuration)	Double transfer + double storage

CRR CDK Configuration

```
const sourceRegion = "us-east-1";
const destRegion = "us-west-2";

// Source bucket (us-east-1) — versioning REQUIRED for replication
const sourceBucket = new s3.Bucket(this, "SourceImages", {
  bucketName: "shopflow-product-images-prod",
  versioned: true, // REQUIRED for CRR
  replicationRules: [{
    id: "ReplicateToDR",
    status: s3.ReplicationRuleStatus.ENABLED,
    destination: {
      bucket: drBucket, // in us-west-2
      storageClass: s3.StorageClass.STANDARD_IA, // save cost in DR
    },
    filter: {prefix: "products/"}, // only replicate product images
    deleteMarkerReplication: false, // do NOT replicate deletes
  }],
});

// Enable S3 Replication Time Control (RTC) for 99.99% replication in 15min
// (required for SLA compliance — additional $0.015/GB replicated)
```

■ Disable deleteMarkerReplication unless you specifically need deletes replicated. With it enabled, a developer accidentally running "delete all" in production would propagate to your DR bucket. Keep delete protection in DR bucket as an independent safety layer.

ShopFlow's React frontend is a Single Page Application (SPA) deployed to S3 and served via CloudFront. The deployment pipeline runs on CodePipeline, invalidates CloudFront cache on each deploy, and uses immutable hashed filenames for JS/CSS to enable aggressive 1-year caching.

ShopFlow SPA Deploy + Serve Architecture

BUILD: CodePipeline → CodeBuild runs `npm ci && npm run build`

Output: `/dist/index.html` (no hash), `/dist/assets/main.abc123.js` (hashed)

Cache control strategy: `index.html=no-cache`, `assets=immutable 1yr`

DEPLOY: CodeBuild syncs to S3 spa bucket

`aws s3 sync dist/ s3://shopflow-spa-assets-prod/ --delete`

`aws s3 cp dist/index.html s3://shopflow-spa-assets-prod/index.html`

`--cache-control "no-cache, no-store, must-revalidate"`

INVALIDATE: CodeBuild creates CF invalidation

`aws cloudfront create-invalidation --paths "/index.html" "/sw.js"`

ONLY invalidate non-hashed files — hashed assets self-invalidate via filena

SERVE: CloudFront edge → browser

`index.html`: CF fetches from S3 on every miss (no-cache), ~30ms edge cache

`main.abc123.js`: CF caches 1yr, browser caches 1yr (immutable)

SPA routing: CF error pages 404→`index.html` (React Router handles `/products/`)

CloudFront Error Page for SPA Routing

```
// SPA routing: 404 from S3 → return index.html (React Router takes over)
const distribution = new cloudfront.Distribution(this, "ShopFlowCF", {
  errorResponses: [{
    httpStatus: 403, // S3 returns 403 for missing objects (not 404)
    responseHttpStatus: 200,
    responsePagePath: "/index.html",
    ttl: Duration.seconds(0), // do not cache error responses
  }, {
    httpStatus: 404,
    responseHttpStatus: 200,
    responsePagePath: "/index.html",
    ttl: Duration.seconds(0),
  }],
  // ... rest of distribution config
});
```

■ S3 returns 403 (not 404) for missing objects when Block Public Access is enabled — there's no "file not found" since S3 won't reveal whether the key exists. Map both 403 AND 404 to `/index.html` in CloudFront error responses for reliable SPA routing.

S3 storage costs account for ~\$340/month of ShopFlow's AWS bill. Smart tiering reduces this by 68% without requiring manual lifecycle rule tuning for objects with unknown or changing access patterns.

ShopFlow S3 Monthly Cost — Before vs After

Bucket / Content Type	Volume	Before Optimization	After Tiering	Monthly Saving
Product images (Standard)	4.2 TB	\$96.60/mo	\$60 (IA after 90d, Glacier 1yr)	\$36.60
Order receipts	890 GB	\$20.47/mo	\$6.75 (IA 1yr, Glacier 2yr)	\$13.72
Log archive (Standard)	2.1 TB	\$48.30/mo	\$9.22 (Deep Archive after 30d)	\$39.08
DB backups (Standard)	1.8 TB	\$41.40/mo	\$1.78 (Deep Archive after 90d)	\$39.62
SPA assets (Standard)	220 MB	\$0.005/mo	\$0.005 (already tiny)	\$0
Requests + data transfer	Varies	\$133/mo	\$133 (unchanged)	\$0
Total	~9 TB	\$340/month	\$211/month	\$129/month (38% saved)

S3 Intelligent-Tiering — How It Works

IT Sub-Tier	Activation Condition	Monthly Cost	Retrieval Fee
Frequent Access	Default for new objects	\$0.023/GB (Standard price)	None
Infrequent Access	No access for 30 consecutive days	\$0.0125/GB (IA price)	None
Archive Instant Access	No access for 90 consecutive days	\$0.004/GB (Glacier Instant)	None
Archive Access (optional)	No access for 90-270 days (opt-in)	\$0.0036/GB (Glacier Flexible)	\$0.01/GB
Deep Archive Access (optional)	No access for 180+ days (opt-in)	\$0.00099/GB (Deep Archive)	\$0.02/GB

S3 Data Transfer Cost Optimization

- S3 to CloudFront: \$0.00/GB (free — S3 → CloudFront is always free)
- S3 to internet (direct): \$0.09/GB first 10TB/month — always use CloudFront instead
- S3 to EC2 same region: \$0.00/GB (free within same region)
- S3 to EC2 different region: \$0.09/GB — always use same region for compute and storage
- S3 Cross-Region Replication transfer: \$0.02/GB (us-east-1 → us-west-2)

■ Enable S3 Storage Lens for organization-wide storage analytics. It provides 60+ metrics including access patterns per bucket, incomplete multipart uploads wasting space, and storage class recommendations — free tier covers 14 days of metrics.

- ✓ Enforce S3 Block Public Access at the AWS account level — prevents any future bucket from going public accidentally
- ✓ Use CloudFront OAC (Origin Access Control) instead of OAI (deprecated) or public buckets
- ✓ Set abort-incomplete-multipart-upload lifecycle rule (7 days) on all buckets to prevent orphaned parts
- ✓ Use hashed filenames for static assets (main.abc123.js) — enables 1-year CloudFront cache with no invalidation cost
- ✓ Enable S3 Versioning on critical buckets — protects against accidental deletion and ransomware overwrites
- ✓ Use S3 presigned URLs (15min TTL) for private content downloads — not permanent public URLs
- ✓ Enable S3 server-side encryption (SSE-S3 minimum, SSE-KMS for compliance) on all buckets
- ✓ Use CloudFront with Origin Shield to reduce origin cache-miss requests by additional 60-80%
- Never use S3 presigned URLs for CDN content — CloudFront OAC is faster, cheaper, and more secure

■ Do not use ACLs — use bucket policies only. AWS recommends disabling ACLs completely on all new buckets

■ Do not serve S3 directly (bypass CloudFront) — no edge caching, no WAF, higher data transfer cost (\$0.09/GB vs \$0.00/GB)

■ S3 WAF is only available on CloudFront (not S3 directly) — always put WAF on CloudFront, not the origin

■ Do not over-invalidate CloudFront — each path invalidation after first 1,000/month costs \$0.005. Use versioned filenames instead

Concept	Key Fact
S3 object max size	5 TB per object (5 GB max for single PUT — multipart required above 5GB)
S3 durability	11 nines (99.999999999%) — data replicated across ≥ 3 AZs
S3 consistency	Strong read-after-write for all operations (since December 2020)
CloudFront edge count	600+ edge locations in 100+ cities, 50+ countries
CloudFront to S3 transfer cost	\$0.00/GB — free between AWS services in same account
S3 → internet direct cost	\$0.09/GB — always use CloudFront instead
WAF for CloudFront region	Must create WebACL in us-east-1 (CloudFront is global, tied to us-east-1)
Multipart required at	5 GB (single PUT max). Recommended at 5 MB for retry + parallel benefits
Intelligent-Tiering monitoring fee	\$0.0025/1,000 objects/month — not worth it for objects < 128KB
S3 Glacier Deep Archive price	\$0.00099/GB/month — cheapest durable AWS storage (~\$1/TB/month)
OAC vs OAI	OAC (new): supports SSE-KMS, all S3 APIs. OAI (deprecated): no KMS, limited. Always use OAC
Lambda@Edge vs CF Functions	CF Functions: JS only, 1ms max, cheaper (\$0.10/M). L@Edge: Node/Python, 30s, network access (\$0.10/GB)
S3 CRR requirement	Both source and destination buckets MUST have Versioning enabled

Top Interview Questions

Question	Answer
How does CloudFront OAC work?	OAC uses SigV4 to sign every S3 request with the CloudFront distribution's identity (cloudfront.amazonaws.com)
S3 vs EBS vs EFS — when to use each?	S3: object storage, unlimited scale, REST API, no mounting. Best for files, images, backups, static web content.
How do you make CloudFront serve static content?	S3 as Origin + S3 as CloudFront distribution. Set defaultRootObject=index.html. Map 403/404 error responses to S3.
What is S3 Intelligent-Tiering and how does it work?	Automatically moves objects between Frequent Access (Standard price), Infrequent Access, and Glacier based on access patterns.
How do you handle Black Friday traffic spikes with S3 and CloudFront?	CloudFront caches static content at edge locations globally — no action needed for scale. Pre-warm CloudFront cache with S3 CloudFront.
S3 CRR vs SRR?	CRR (Cross-Region Replication): source and destination in different regions. For DR and compliance. IRR (Intra-Region Replication) is also possible.
How to secure S3 from public access?	Enable S3 Block Public Access at account + bucket level. Use OAC for CloudFront. Use presigned URLs for temporary access.